6
Combinatorics

Combinatorics is the mathematics of counting. There are several basic counting problems that occur repeatedly throughout computer science and programming.

Combinatorics problems are notorious for their reliance on cleverness and insight. Once you look at the problem in the right way, the answer suddenly becomes obvious. This *aha!* phenomenon makes them ideal for programming contests, because the right observation can replace the need to write a complicated program that generates and counts all solutions with one call to a simple formula. It sometimes leads to "off-line" contest solutions. If the resulting computations are tractable only on small integers or are in fact the same for all input, one might be able to compute all possible solutions using (say) a pocket calculator and then write a program to print out the answers on demand. Remember, the judge can't look into your heart or your program to see your intentions – it only checks the results.

6.1 Basic Counting Techniques

Here we review certain basic counting rules and formulas you may have seen but possibly forgotten. In particular, there are three basic counting rules from which many counting formulae are generated. It is important to see which rule applies for your particular problem:

- *Product Rule* — The *product rule* states that if there are $|A|$ possibilities from set A and $|B|$ possibilities from set B, then there are $|A| \times |B|$ ways to combine one from A and one from B. For example, suppose you own 5 shirts and 4 pants. Then there are $5 \times 4 = 20$ different ways you can get dressed tomorrow.

- *Sum Rule* — The *sum rule* states that if there are $|A|$ possibilities from set A and $|B|$ possibilities from set B, then there are $|A| + |B|$ ways for either A *or* B to occur – assuming the elements of A and B are distinct. For example, given that you own 5 shirts and 4 pants and the laundry ruined one of them, there are 9 possible ruined items.[1]

- *Inclusion-Exclusion Formula* — The sum rule is a special case of a more general formula when the two sets can overlap, namely,

$$|A \cup B| = |A| + |B| - |A \cap B|$$

  For example, let A represent the set of colors of my shirts and B the colors of my pants. Via inclusion-exclusion, I can calculate the total number of colors given the number of color-matched garments or vice versa. The reason this works is that summing the sets double counts certain possibilities, namely, those occurring in both sets. The inclusion-exclusion formula generalizes to three sets and beyond in a natural way:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Double counting is a slippery aspect of combinatorics, which can make it difficult to solve problems via inclusion-exclusion. Another powerful technique is establishing a bijection. A *bijection* is a one-to-one mapping between the elements of one set and the elements of another. Whenever you have such a mapping, counting the size of one of the sets automatically gives you the size of the other set.

For example, if we count the number of pants currently being worn in a given class, and can assume that all students wears pants, then this tells us the number of people in the class. It works because there is a one-to-one mapping between pants and people, and would break if we exchanged pants for socks or removed the dress code and allowed people to wear skirts instead.

Exploiting bijections requires us to have a repertoire of sets which we know how to count, so we can map other objects to them. Basic combinatorial objects you should be familiar with include the following. It is useful to have a feeling for how fast the number of objects grows, to know when exhaustive search breaks down as a possible technique:

- *Permutations* — A *permutation* is an arrangement of n items, where every item appears exactly once. There are $n! = \prod_{i=1}^{n} i$ different permutations. The $3! = 6$ permutations of three items are 123, 132, 213, 231, 312, and 321. For $n = 10$, $n! = 3,628,800$, so we start to approach the limits of exhaustive search.

- *Subsets* — A *subset* is a selection of elements from n possible items. There are 2^n distinct subsets of n things. Thus there are $2^3 = 8$ subsets of three items, namely, 1, 2, 3, 12, 13, 23, 123, and the empty set: never forget the empty set. For $n = 20$, $2^n = 1,048,576$, so we start to approach the limits of exhaustive search.

- *Strings* — A *string* is a sequence of items which are drawn *with repetition*. There are m^n distinct sequences of n items drawn from m items. The 27 length-3 strings

[1] This is not true in practice, because the ruined item is certain to be your favorite of the bunch.

on 123 are 111, 112, 113, 121, 122, 123, 131, 132, 133, 211, 212, 213, 221, 222, 223, 231, 232, 233, 311, 312, 313, 321, 322, 323, 331, 332, and 333. The number of binary strings of length n is identical to the number of subsets of n items (why?), and the number of possibilities increases even more rapidly with larger m.

6.2 Recurrence Relations

Recurrence relations make it easy to count a variety of recursively defined structures. Recursively defined structures include trees, lists, well-formed formulae, and divide-and-conquer algorithms – so they lurk wherever computer scientists do.

What is a recurrence relation? It is an equation which is defined in terms of itself. Why are they good things? Because many natural functions are easily expressed as recurrences! Any polynomial can be represented by a recurrence, including the linear function:

$$a_n = a_{n-1} + 1, a_1 = 1 \longrightarrow a_n = n$$

Any exponential can be represented by a recurrence:

$$a_n = 2a_{n-1}, a_1 = 2 \longrightarrow a_n = 2^n$$

Finally, certain weird but interesting functions which are not easily represented using conventional notation can be described by recurrences:

$$a_n = na_{n-1}, a_1 = 1 \longrightarrow a_n = n!$$

Thus recurrence relations are a very versatile way to represent functions. It is often easy to find a recurrence as the answer to a counting problem. *Solving* the recurrence to get a nice closed form can be somewhat of an art, but as we shall see, computer programs can easily evaluate the value of a given recurrence even without the existence of a nice closed form.

6.3 Binomial Coefficients

The most important class of counting numbers are the *binomial coefficients*, where $\binom{n}{k}$ counts the number of ways to choose k things out of n possibilities. What do they count?

- *Committees* — How many ways are there to form a k-member committee from n people? By definition, $\binom{n}{k}$ is the answer.
- *Paths Across a Grid* — How many ways are there to travel from the upper-left corner of an $n \times m$ grid to the lower-right corner by walking only down and to the right? Every path must consist of $n + m$ steps, n downward and m to the right. Every path with a different set of downward moves is distinct, so there are $\binom{n+m}{n}$ such sets/paths.

- *Coefficients of* $(a+b)^n$ — Observe that

$$(a+b)^3 = 1a^3 + 3a^2b + 3ab^2 + 1b^3$$

What is the coefficient of the term $a^k b^{n-k}$? Clearly $\binom{n}{k}$, because it counts the number of ways we can choose the k a-terms out of n possibilities.

- *Pascal's Triangle* — No doubt you played with this arrangement of numbers in high school. Each number is the sum of the two numbers directly above it:

```
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5  10  10  5   1
```

Why did you or Pascal care? Because this table constructs the binomial coefficients! The $(n+1)$st row of the table gives the values $\binom{n}{i}$ for $0 \leq i \leq n$. The neat thing about the triangle is how it reveals certain interesting identities, such that the sum of the entries on the $(n+1)$st row equals 2^n.

How do you compute the binomial coefficients? First, $\binom{n}{k} = n!/((n-k)!k!)$, so in principle you can compute them straight from factorials. However, this method has a serious drawback. Intermediate calculations can easily cause arithmetic overflow even when the final coefficient fits comfortably within an integer.

A more stable way to compute binomial coefficients is using the recurrence relation implicit in the construction of Pascal's triangle, namely, that

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Why does this work? Consider whether the nth element appears in one of the $\binom{n}{k}$ subsets of k elements. If so, we can complete the subset by picking $k-1$ other items from the other $n-1$. If not, we must pick all k items from the remaining $n-1$. There is no overlap between these cases, and all possibilities are included, so the sum counts all k-subsets.

No recurrence is complete without basis cases. What binomial coefficient values do we know without computing them? The left term of the sum eventually drives us down to $\binom{n-k}{0}$. How many ways are there to choose 0 things from a set? Exactly one, the empty set. If this is not convincing, then it is equally good to accept that $\binom{m}{1} = m$. The right term of the sum drives us up to $\binom{k}{k}$. How many ways are there to choose k things from a k-element set? Exactly one, the complete set. Together with the recurrence these basis cases define the binomial coefficients on all interesting values.

The best way to evaluate such a recurrence is to build a table of all possible values, at least up to the size that you are interested in. Study the function below to see how we did it.

```
#define MAXN    100             /* largest n or m */

long binomial_coefficient(n,m)
int n,m;                        /* computer n choose m */
{
        int i,j;                /* counters */
        long bc[MAXN][MAXN];    /* table of binomial coefficients */

        for (i=0; i<=n; i++) bc[i][0] = 1;

        for (j=0; j<=n; j++) bc[j][j] = 1;

        for (i=1; i<=n; i++)
                for (j=1; j<i; j++)
                        bc[i][j] = bc[i-1][j-1] + bc[i-1][j];

        return( bc[n][m] );
}
```

Such programs to evaluate recurrences are the foundation for dynamic programming, an algorithmic technique we will study in Chapter 11.

6.4 Other Counting Sequences

There are several other counting sequences which repeatedly emerge in applications, and which are easily computed using recurrence relations. The wise combinatorialist keeps them in mind whenever they count:

- *Fibonacci numbers* — Defined by the recurrence $F_n = F_{n-1} + F_{n-2}$ and the initial values $F_0 = 0$ and $F_1 = 1$, they emerge repeatedly because this is perhaps the simplest interesting recurrence relation. The first several values are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... The Fibonacci numbers lend themselves to an amazing variety of mathematical identities, and are just fun to play with. They have the following hard-to-guess but simple-to-derive closed form:

$$F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right)$$

  This closed form has certain important implications. Since $(1-\sqrt{5})/2$ is between 0 and 1, raising it to any power leaves a number in this range. Thus the first term, ϕ^n where $\phi = (1+\sqrt{5})/2$ is the driving quantity, and can be used to estimate F_n to within plus or minus 1.

- *Catalan Numbers* — The recurrence and associated closed form

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k} = \frac{1}{n+1}\binom{2n}{n}$$

defines the *Catalan numbers*, which occur in a surprising number of problems in combinatorics. The first several terms are $2, 5, 14, 42, 132, 429, 1430, \ldots$ when $C_0 = 1$.

How many ways are there to build a balanced formula from n sets of left and right parentheses? For example, there are five ways to do it for $n = 3$: ((())), ()(()), (())(), (()()), and ()()(). The leftmost parenthesis l matches some right parenthesis r, which must partition the formula into two balanced pieces, the part between l and r, and the part to the right of r. If the left part contains k pairs, the right part must contain $n - k - 1$ pairs, since l,r represent one pair. Both of these subformulas must be well formed, which leads to the recurrence:

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$$

and we have the Catalan numbers.

The exact same reasoning arises in counting the number of triangulations of a convex polygon, counting the number of rooted binary trees on $n+1$ leaves, and counting the number of paths across a lattice which do not rise above the main diagonal. The Catalan numbers have the nice closed form $C_n = \frac{1}{n+1}\binom{2n}{n}$.

- *Eulerian Numbers* — The *Eulerian* numbers $\left\langle {n \atop k} \right\rangle$ count the number of permutations of length n with exactly k ascending sequences or *runs*. A recurrence can be formulated by considering each permutation p of $1, ..., n-1$. There are n places to insert element n, and each either splits an existing run of p or occurs immediately after the last element of an existing run, thus preserving the run count. Thus $\left\langle {n \atop k} \right\rangle = k\left\langle {n-1 \atop k} \right\rangle + (n-k+1)\left\langle {n-1 \atop k-1} \right\rangle$. Can you construct the eleven permutations of length four with exactly two runs?

- *Stirling Numbers* — There are two different types of Stirling numbers. The first, $\left[{n \atop k} \right]$, counts the number of permutations on n elements with exactly k cycles. To formulate the recurrence, observe the nth element either forms a singleton cycle or it doesn't. If it does, there are $\left[{n-1 \atop k-1} \right]$ ways to arrange the rest of the elements to form $k-1$ cycles. If not, the nth element can be inserted in every possible position of every cycle of the $\left[{n-1 \atop k} \right]$ ways to make k cycles out of $n-1$ elements. Thus

$$\left[{n \atop k} \right] = \left[{n-1 \atop k-1} \right] + (n-1)\left[{n-1 \atop k} \right]$$

There are 11 permutations of four elements with exactly two cycles.

- *Set Partitions* — The second kind of Stirling number $\{{n \atop k}\}$ counts the number of ways to partition n items into k sets. For example, there are seven ways to partition four items into exactly two subsets: (1)(234), (12)(34), (13)(24), (14)(23), (123)(4), (124)(3) and (134)(2). The nth item can be inserted into any of the k subsets of an $n-1$-part partition or it forms a singleton set. Thus by a similar argument to that of the other Stirling numbers they are defined by the recurrence $\{{n \atop k}\} = k\{{n-1 \atop k}\} + \{{n-1 \atop k}\}$. The special case of $\{{n \atop 2}\} = 2^{n-1} - 1$, since any proper subset of the elements 2 to n can be unioned with (1) to define the set partition. The second part of the partition consists of exactly the elements not in this first part.

- *Integer Partitions* — An integer partition of n is an unordered set of positive integers which add up to n. For example, there are seven partitions of 5, namely, (5), (4, 1), (3, 2), (3, 1, 1), (2, 2, 1), (2, 1, 1, 1), and (1, 1, 1, 1, 1). The easiest way to count them is to define a function $f(n, k)$ giving the number of integer partitions of n with largest part at most k. In any acceptable partition the largest part either does or does not reach with limit, so $f(n, k) = f(n-k, k) + f(n, k-1)$. The basis cases are $f(1, 1) = 1$ and $f(n, k) = 0$ whenever $k > n$.

The interested student should read [GKP89] for more on these and other interesting counting sequences. It is also worth visiting Sloane's *Handbook of Integer Sequences* on the web at *http://www.research.att.com/~njas/sequences/* to help identify virtually any interesting sequence of integers.

6.5 Recursion and Induction

Mathematical induction provides a useful tool to solve recurrences. When we first learned about mathematical induction in high school it seemed like complete magic. You proved the formula for some basis case like 1 or 2, then *assumed* it was true all the way to $n-1$ before proving it was true for general n using the assumption. That was a proof? Ridiculous!

When we first learned the programming technique of recursion in college it also seemed like complete magic. Your program tested whether the input argument was some basis case like 1 or 2. If not, you solved the bigger case by breaking it up into pieces and *calling the subprogram itself* to solve these pieces. That was a program? Ridiculous!

The reason both seemed like magic is because recursion *is* mathematical induction! In both, we have general and boundary conditions, with the general condition breaking the problem into smaller and smaller pieces. The *initial* or boundary condition terminates the recursion. Once you understand either recursion or induction, you should be able to turn it around to see why the other one also works.

A powerful way to solve recurrence relations is to guess a solution and then prove it by induction. When trying to guess a solution, it pays to tabulate small values of the function and stare at them until you see a pattern.

For example, consider the following recurrence relation:

$$T_n = 2T_{n-1} + 1, T_0 = 0$$

Building a table of values yields the following:

n	0	1	2	3	4	5	6	7
T_n	0	1	3	7	15	31	63	127

Can you guess what the solution is? You should notice that things look like they are doubling each time, no surprise considering the formula. But it is not quite 2^n. By playing around with variations of this function, you should be able to stumble on the conjecture that $T_n = 2^n - 1$. To finish the job, we must prove this conjecture, using the three steps of induction:

1. Show that the basis is true: $T_0 = 2^0 - 1 = 0$.
2. Now assume it is true for T_{n-1}.
3. Use this assumption to complete the argument:

$$T_n = 2T_{n-1} + 1 = 2(2^{n-1} - 1) + 1 = 2^n - 1$$

Guessing the solution is usually the hard part of the job, and where the art and experience comes in. The key is playing around with small values for insight, and having some feel for what kind of closed form the answer will be.

6.6 Problems

6.6.1 How Many Fibs?

PC/UVa IDs: 110601/10183, **Popularity:** B, **Success rate:** average **Level:** 1

Recall the definition of the Fibonacci numbers:

$$\begin{aligned} f_1 &:= 1 \\ f_2 &:= 2 \\ f_n &:= f_{n-1} + f_{n-2} \qquad (n \geq 3) \end{aligned}$$

Given two numbers a and b, calculate how many Fibonacci numbers are in the range $[a, b]$.

Input

The input contains several test cases. Each test case consists of two non-negative integer numbers a and b. Input is terminated by $a = b = 0$. Otherwise, $a \leq b \leq 10^{100}$. The numbers a and b are given with no superfluous leading zeros.

Output

For each test case output on a single line the number of Fibonacci numbers f_i with $a \leq f_i \leq b$.

Sample Input

```
10 100
1234567890 9876543210
0 0
```

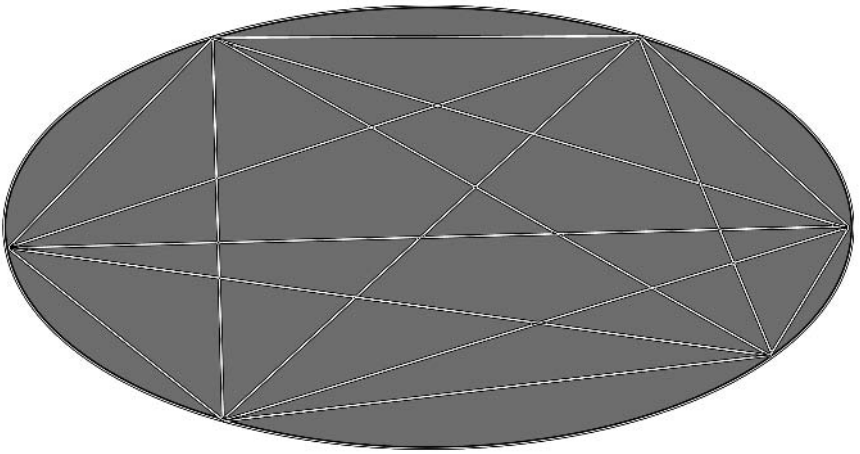
Sample Output

```
5
4
```

6.6.2 How Many Pieces of Land?

PC/UVa IDs: 110602/10213, **Popularity:** B, **Success rate:** average **Level:** 2

You are given an elliptical-shaped land and you are asked to choose n arbitrary points on its boundary. Then you connect each point with every other point using straight lines, forming $n(n-1)/2$ connections. What is the maximum number of pieces of land you will get by choosing the points on the boundary carefully?



Dividing the land when $n = 6$.

Input

The first line of the input file contains one integer s $(0 < s < 3,500)$, which indicates how many input instances there are. The next s lines describe s input instances, each consisting of exactly one integer n $(0 \leq n < 2^{31})$.

Output

For each input instance output the maximum possible number pieces of land defined by n points, each printed on its own line.

Sample Input	*Sample Output*
4 1 2 3 4	1 2 4 8

6.6.3 Counting

PC/UVa IDs: 110603/10198, **Popularity:** B, **Success rate:** high **Level:** 2

Gustavo knows how to count, but he is just now learning how to write numbers. He has already learned the digits 1, 2, 3, and 4. But he does not yet realize that 4 is different than 1, so he thinks that 4 is just another way to write 1.

He is having fun with a little game he created: he makes numbers with the four digits that he knows and sums their values. For example:

`132 = 1 + 3 + 2 = 6`
`112314 = 1 + 1 + 2 + 3 + 1 + 1 = 9` (remember that Gustavo thinks that `4 = 1`)

Gustavo now wants to know how many such numbers he can create whose sum is a number n. For $n = 2$, he can make 5 numbers: 11, 14, 41, 44, and 2. (He knows how to count up beyond five, just not how to write it.) However, he can't figure out this sum for n greater than 2, and asks for your help.

Input

Input will consist of an arbitrary number of integers n such that $1 \leq n \leq 1,000$. You must read until you reach the end of file.

Output

For each integer read, output an single integer on a line stating how many numbers Gustavo can make such that the sum of their digits is equal to n.

Sample Input

```
2
3
```

Sample Output

```
5
13
```

6.6.4 Expressions

PC/UVa IDs: 110604/10157, **Popularity:** C, **Success rate:** average **Level:** 2

Let X be the set of *correctly built parenthesis expressions*. The elements of X are strings consisting only of the characters "(" and ")", defined as follows:

- The empty string belongs to X.
- If A belongs to X, then (A) belongs to X.
- If both A and B belong to X, then the concatenation AB belongs to X.

For example, the strings ()(())() and ((()())) are correctly built parenthesis expressions, and therefore belong to the set X. The expressions ()))(() and))(() are not correctly built parenthesis expressions and are thus not in X.

The *length* of a correctly built parenthesis expression E is the number of single parenthesis (characters) in E. The *depth* $D(E)$ of E is defined as follows:

$$D(E) = \begin{cases} 0 & \text{if } E \text{ is empty} \\ D(A)+1 & \text{if } E = (A), \text{ and } A \text{ is in X} \\ \max(D(A), D(B)) & \text{if } E = AB, \text{ and } A, B \text{ are in X} \end{cases}$$

For example, ()(())() has length 8 and depth 2. Write a program which reads in n and d and computes the number of correctly built parenthesis expressions of length n and depth d.

Input

The input consists of pairs of integers n and d, with at most one pair per line and $2 \leq n \leq 300$, $1 \leq d \leq 150$. The input may contain empty lines, which you don't need to consider.

Output

For every pair of integers in the input, output a single integer on one line – the number of correctly built parenthesis expressions of length n and depth d.

Sample Input	*Sample Output*
6 2	3
300 150	1

Note: The three correctly built parenthesis expressions of length 6 and depth 2 are (())(), ()(()), and (()()).

6.6.5 Complete Tree Labeling

PC/UVa IDs: 110605/10247, **Popularity:** C, **Success rate:** average **Level:** 2

A complete k-ary tree is a k-ary tree in which all leaves have same depth and all internal nodes have degree or (equivalently) branching factor k. It is easy to determine the number of nodes of such a tree.

Given the depth and branching factor of such a tree, you must determine in how many different ways you can number the nodes of the tree so that the label of each node is less that that of its descendants. This is the property which defines the binary heap priority queue data structure for $k = 2$. In numbering a tree with N nodes, assume you have the labels $(1, 2, 3, \ldots, N-1, N)$ available.

Input

The input file will contain several lines of input. Each line will contain two integers k and d. Here $k > 0$ is the branching factor of the complete k-ary tree and $d > 0$ is the depth of the complete k-ary tree. Your program must work for all pairs such that $k \times d \leq 21$.

Output

For each line of input, produce one line of output containing an integer counting the number of ways the k-ary tree can be labeled, maintaining the constraints described above.

Sample Input

```
2 2
10 1
```

Sample Output

```
80
3628800
```

6.6.6 The Priest Mathematician

PC/UVa IDs: 110606/10254, **Popularity:** C, **Success rate:** high **Level:** 2

The ancient folklore behind the *"Towers of Hanoi"* puzzle is quite well known. A more recent legend tells us that once the Brahmin monks discovered how long it would take to finish transferring the 64 discs from the needle which they were on to one of the other needles, they decided to find a faster strategy and be done with it.



The Four Needle (Peg) Tower of Hanoi

One of the priests at the temple informed his colleagues that they could achieve the transfer in single afternoon at a one disc-per-second rhythm by using an additional needle. He proposed the following strategy:

- First move the topmost discs (say the top k discs) to one of the spare needles.
- Then use the standard three needles strategy to move the remaining $n - k$ discs (for a general case with n discs) to their destination.
- Finally, move the top k discs into their final destination using the four needles.

He calculated the value of k which minimized the number of movements and found that 18,433 transfers would suffice. Thus they could spend just 5 hours, 7 minutes, and 13 seconds with this scheme versus over $500,000$ million years without the additional needle!

Try to follow the clever priest's strategy and calculate the number of transfers using four needles, where the priest can move only one disc at a time and must place each disc on a needle such that there is no smaller disc below it. Calculate the k that minimizes the number of transfers under this strategy.

Input

The input file contains several lines of input. Each line contains a single integer $0 \leq N \leq 10,000$ giving the number of disks to be transferred. Input is terminated by end of file.

Output

For each line of input produce one line of output which indicates the number of movements required to transfer the N disks to the final needle.

Sample Input	*Sample Output*
1	1
2	3
28	769
64	18433

6.6.7 Self-describing Sequence

PC/UVa IDs: 110607/10049, **Popularity:** C, **Success rate:** high **Level:** 2

Solomon Golomb's *self-describing sequence* $\langle f(1), f(2), f(3), \ldots \rangle$ is the only non-decreasing sequence of positive integers with the property that it contains exactly $f(k)$ occurrences of k for each k. A few moment's thought reveals that the sequence must begin as follows:

n	1	2	3	4	5	6	7	8	9	10	11	12
$f(n)$	1	2	2	3	3	4	4	4	5	5	5	6

In this problem you are expected to write a program that calculates the value of $f(n)$ given the value of n.

Input

The input may contain multiple test cases. Each test case occupies a separate line and contains an integer n ($1 \leq n \leq 2,000,000,000$). The input terminates with a test case containing a value `0` for n and this case must not be processed.

Output

For each test case in the input, output the value of $f(n)$ on a separate line.

Sample Input

```
100
9999
123456
1000000000
0
```

Sample Output

```
21
356
1684
438744
```

6.6.8 Steps

PC/UVa IDs: 110608/846, **Popularity:** A, **Success rate:** high **Level:** 2

Consider the process of stepping from integer x to integer y along integer points of the straight line. The length of each step must be non-negative and can be one bigger than, equal to, or one smaller than the length of the previous step.

What is the minimum number of steps in order to get from x to y? The length of both the first and the last step must be 1.

Input

The input begins with a line containing n, the number of test cases. Each test case that follows consists of a line with two integers: $0 \leq x \leq y < 2^{31}$.

Output

For each test case, print a line giving the minimum number of steps to get from x to y.

Sample Input

```
3
45 48
45 49
45 50
```

Sample Output

```
3
3
4
```

6.7 Hints

6.1 Can the closed form for F_n be used to minimize the need for arbitrary-precision arithmetic?

6.2 Can you find a recurrence for the desired quantity?

6.3 Can you find a recurrence for the desired sum?

6.4 Can you formulate a recurrence, maybe a two-parameter version of the Catalan numbers?

6.5 Can you find a recurrence for the desired quantity?

6.7 Can you explicitly build the sequence, or must you do something more clever because of limited memory?

6.8 What kind of step sequences are defined by optimal solutions?

6.8 Notes

6.6 Although the problem asks for a fast way to solve the four-peg Tower of Hanoi problem under this strategy, it is not known that this strategy is in fact optimal! See [GKP89] for more discussion.